



Running Poolora on an Android device

Step-by-step: connect any Android phone or emulator with adb, start the backend, and install and run the app for manual testing.

The short version

Once everything is installed, these five commands run the app on a connected phone:

```
adb devices          # the phone must show "device"
cd backend && npm run dev # API on http://localhost:5002
adb reverse tcp:5002 tcp:5002 # phone localhost:5002 -> laptop API
adb reverse tcp:8081 tcp:8081 # phone localhost:8081 -> Metro bundler
cd frontend && npx expo run:android # build, install and open the app
```

The rest of this guide explains each step, other ways to connect, and what to do when something goes wrong.

1. What you need

Item	Version	Check with
Node.js and npm	18 or newer (tested with 22)	node -v
Java JDK	17	java -version
Android SDK with platform-tools (adb)	Installed by Android Studio, or the command-line tools	adb version
Docker	Any recent version, for MongoDB and Redis	docker --version
Android phone or emulator	Android 7.0 (API 24) or newer; the app targets API 36	Settings > About phone
USB cable	One that carries data, not only charge	

Make sure the Android SDK is on your path so Gradle can find it, for example in ~/.bashrc or ~/.zshrc:

```
export ANDROID_HOME=$HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/platform-tools
```

2. Prepare the phone (once per phone)

- 1 Open **Settings > About phone** (on Samsung: **About phone > Software information**) and tap **Build number** seven times until it says developer mode is on.
- 2 Open **Settings > Developer options** and turn on **USB debugging**.
- 3 Connect the phone with the USB cable. If the phone asks what to use USB for, choose **File transfer**.
- 4 A prompt appears on the phone: **Allow USB debugging?** Tick **Always allow from this computer** and tap **Allow**.

Check that adb sees it:

```
adb devices -l
# List of devices attached
# RZ8W5021XNV    device usb:3-3 product:a04snnxx model:SM_A047F
```

adb shows	Meaning	Fix
device	Connected and authorised	Nothing to do
unauthorized	The phone has not accepted this computer	Unlock the phone and accept the prompt. If it never appears: Developer options > Revoke USB debugging authorisations, unplug, plug in again
offline	adb lost the connection	adb kill-server, then adb start-server; replug the cable
(nothing)	No data connection	Try another cable or port; set USB mode to File transfer; on Linux add a udev rule for the phone

Without a cable: wireless debugging (Android 11 and newer)

Put the laptop and phone on the same Wi-Fi. In **Developer options > Wireless debugging**, turn it on and tap **Pair device with pairing code**. The phone shows an address, a port and a six-digit code.

```
# pairing address and port from the pairing pop-up; type the code when asked
adb pair 192.168.1.23:37115
# address and port shown on the Wireless debugging screen
adb connect 192.168.1.23:41637
adb devices                # shows 192.168.1.23:41637 device
```

Everything else in this guide, including adb reverse, works the same over Wi-Fi.

An emulator instead of a phone

In Android Studio, open **Device Manager**, create a virtual device with API 24 or newer, and start it. It appears in adb devices as emulator-5554 and is used exactly like a phone.

3. Start the backend

The app needs the API running. Copy the example settings once and fill in your own values (MongoDB, JWT secrets, Firebase, and Razorpay if you test card payments):

```
cd backend
cp .env.example .env                # then edit .env
# Firebase service account: secrets/firebase-service-account.json (see docs/SECRETS.md)
```

Option A: backend on your laptop, databases in Docker (best for development)

```
cd backend
docker compose up -d mongo redis    # MongoDB on localhost:27018, Redis on localhost:6379
# in .env for this option:
# MONGO_URI=mongodb://admin:<password>@localhost:27018/mobility?authSource=admin
# REDIS_HOST=localhost
npm install                          # first time only
npm run dev                          # restarts itself when you change the code
```

A MongoDB Atlas connection string in MONGO_URI works too, and then only Redis is needed from Docker.

Option B: everything in Docker

```
cd backend
docker compose up -d --build          # API, ML service, MongoDB, Redis and Kafka
```

Either way, check the API is up before touching the phone:

```
curl http://localhost:5002/health
# {"status":"success","code":200,"data":{"service":"mobility-backend",...}}
```

Kafka is optional. Without it the log repeats "Kafka not reachable yet; retrying" and events are handled inside the API, which is fine for testing.

4. Let the phone reach the laptop

The app talks to two services on the laptop: the API on port **5002** and, for development builds, the Metro bundler on port **8081**, which serves the JavaScript. Pick one of these methods.

Method 1: adb reverse (recommended, works with any adb device)

This makes localhost on the phone lead to the laptop, over the cable or wireless debugging. The app already uses `http://localhost:5002` in development, so nothing needs changing.

```
adb reverse tcp:5002 tcp:5002
adb reverse tcp:8081 tcp:8081
adb reverse --list          # both should be listed
```

Run these again whenever the phone is unplugged, reconnected, restarted, or adb restarts (installing the app can restart adb). If the app suddenly says "Unable to connect to the server", this is almost always the reason.

Method 2: same Wi-Fi, using the laptop's address

Find the laptop's address (ip addr on Linux, ipconfig on Windows), then set it in `frontend/.env` and restart Metro:

```
REACT_NATIVE_API_BASE_URL=http://192.168.1.10:5002
```

The laptop firewall must allow ports 5002 and 8081. This does not work with Option B above, because Docker only publishes the API on 127.0.0.1.

5. Build, install and run the app

```
cd frontend
npm install          # first time only
npx expo run:android # builds the Android app, installs it and starts Metro
```

The first build downloads Gradle and the Android libraries and takes 5 to 15 minutes; later builds take about a minute. When it finishes, the app opens on the phone and Metro keeps running in the terminal.

Several devices connected

`npx expo run:android` asks which device to use when more than one is connected. To pick one directly, pass the device **name** (not the serial number), for example:

```
npx expo run:android --device "SM_A047F"
# adb commands take the serial number instead:
adb -s RZ8W5021XNV reverse tcp:5002 tcp:5002
```

Next time: no rebuild needed

The installed app is a development build. Rebuild only when native code or packages change. Otherwise start Metro and open the app:

```
cd frontend
npx expo start --dev-client      # then press "a", or tap the Poolora icon on the phone
```

Code changes reload by themselves. Press **r** in the Metro terminal to reload by hand, or open the developer menu on the phone by shaking it or running adb shell input keyevent 82.

Installing on a phone without building on your laptop

Build an APK once and install it on any number of phones:

```
cd frontend/android
# output: app/build/outputs/apk/debug/app-debug.apk (it still needs Metro)
./gradlew assembleDebug
adb -s <serial> install -r app/build/outputs/apk/debug/app-debug.apk
```

A **release** APK (./gradlew assembleRelease) contains the JavaScript, so it runs without Metro, but it must be built with REACT_NATIVE_API_BASE_URL set to an **https** address: release builds refuse plain http, and the app stops at start-up if the address is missing. This project signs release builds with the debug key, which is fine for testing but not for the Play Store.

6. Useful while testing

Task	How
Test a full trip on one phone	Settings > Testing > Simulate a ride (as rider or as driver). A bot plays the other person. Available when the backend is not in production mode.
See the app's logs	In the Metro terminal, or: adb logcat -s ReactNativeJS
See the API's logs	The terminal running npm run dev, or: docker compose logs -f app
Take a screenshot	adb exec-out screencap -p > screen.png
Record the screen	adb shell screenrecord /sdcard/test.mp4 (Ctrl+C to stop), then adb pull /sdcard/test.mp4
Type text into the phone	adb shell input text "Indiranagar"
Start again from a clean app	adb shell pm clear com.poolora.app (signs you out and clears saved data)
Remove the app	adb uninstall com.poolora.app
Open the web admin	cd admin-web, npm run dev, then open the address it prints in a browser

Some features need services that may not be set up locally: text messages need Twilio (TWILIO_* in backend/.env), emails need an SMTP server (SMTP_*), and card or UPI payments need Razorpay keys. Without them the app says the feature is unavailable; wallet payments and the ride simulator work without any of them.

7. Troubleshooting

Problem	Likely cause	Fix
"Unable to connect to the server" in the app	The port forwarding dropped, or the API is not running	curl http://localhost:5002/health on the laptop; then run both adb reverse commands again and pull down or tap retry
Red screen: "Unable to load script"	The phone cannot reach Metro	Start Metro (npx expo start --dev-client) and run adb reverse tcp:8081 tcp:8081
CommandError: Could not find device with name	--device was given the serial number	Pass the device name (the model shown by adb devices -l), or leave --device out
SDK location not found	Gradle cannot find the Android SDK	Set ANDROID_HOME, or create frontend/android/local.properties with sdk.dir=/path/to/Android/Sdk
Unsupported class file major version	Wrong Java version	Use JDK 17 (set JAVA_HOME)
INSTALL_FAILED_UPDATE_INCOMPATIBLE	A copy signed with another key is installed	adb uninstall com.poolora.app, then install again
Google sign-in fails	This computer's debug key is not registered in Firebase	Add the SHA-1 from ./gradlew signingReport to the Firebase Android app, or sign in with a phone number
Map is blank	No internet on the phone	Map tiles come from OpenFreeMap over the internet; check the phone's data or Wi-Fi
Port 8081 or 5002 already in use	An old Metro or API process	Stop it (lsof -i :8081 on Linux or macOS), or run Metro on another port with --port 8082 and reverse that port

8. Checklist before a demo or test session

- Docker is running, and MongoDB and Redis are up
- curl http://localhost:5002/health returns status success
- adb devices shows the phone as **device**
- adb reverse --list shows 5002 and 8081
- Metro is running (npx expo start --dev-client), or you installed a release build
- The phone has internet access for maps and sign-in